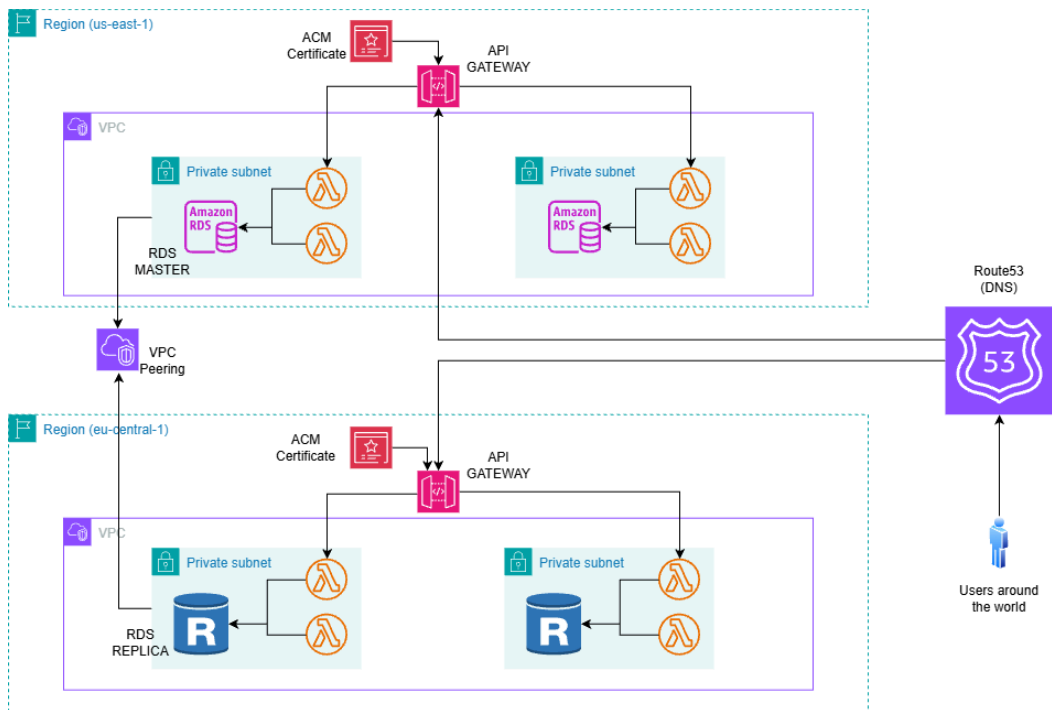


GLOBAL LATENCY IN AWS

Measuring Single-Region and Multi-Region Architecture Latency



1. Introduction

The goal of this laboratory was to understand how the geographic location of a user affects the latency of a backend hosted in AWS. Instead of looking at latency only as a theoretical concept, I wanted to measure it directly and compare how the same API behaves when requests originate from different parts of the world.

I was also interested in validating common multi-region design ideas with real numbers. For example, I already knew the theory behind using read replicas to reduce read latency for distant users, but I wanted to see how much difference it actually makes in practice.

The experiment therefore focuses on measuring end-to-end latency across several global locations and comparing a single-region architecture with a multi-region setup that introduces regional compute, a read replica, and latency-based routing.

2. Objectives

- How much does geographic distance affect API latency?
- How much can a regional read replica improve read latency?
- Does moving API Gateway and Lambda closer to users improve writes if the primary database remains remote?
- How does Route 53 latency-based routing behave with this architecture?
- Is a two-region architecture enough to provide low latency globally?

3. Architecture

The laboratory was divided into two main architectures: a single-region setup and a multi-region setup. The objective was to compare how the same backend behaves when all components are centralized in one region versus when reads are distributed closer to users.

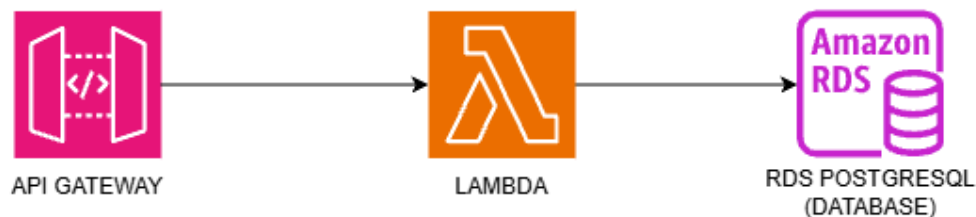
3.1. Single-region

The initial architecture was deployed entirely in N. Virginia (us-east-1).

It included:

- API Gateway
- AWS Lambda
- An Amazon RDS PostgreSQL primary instance

All reads and writes were handled in the same region, regardless of where the request originated.



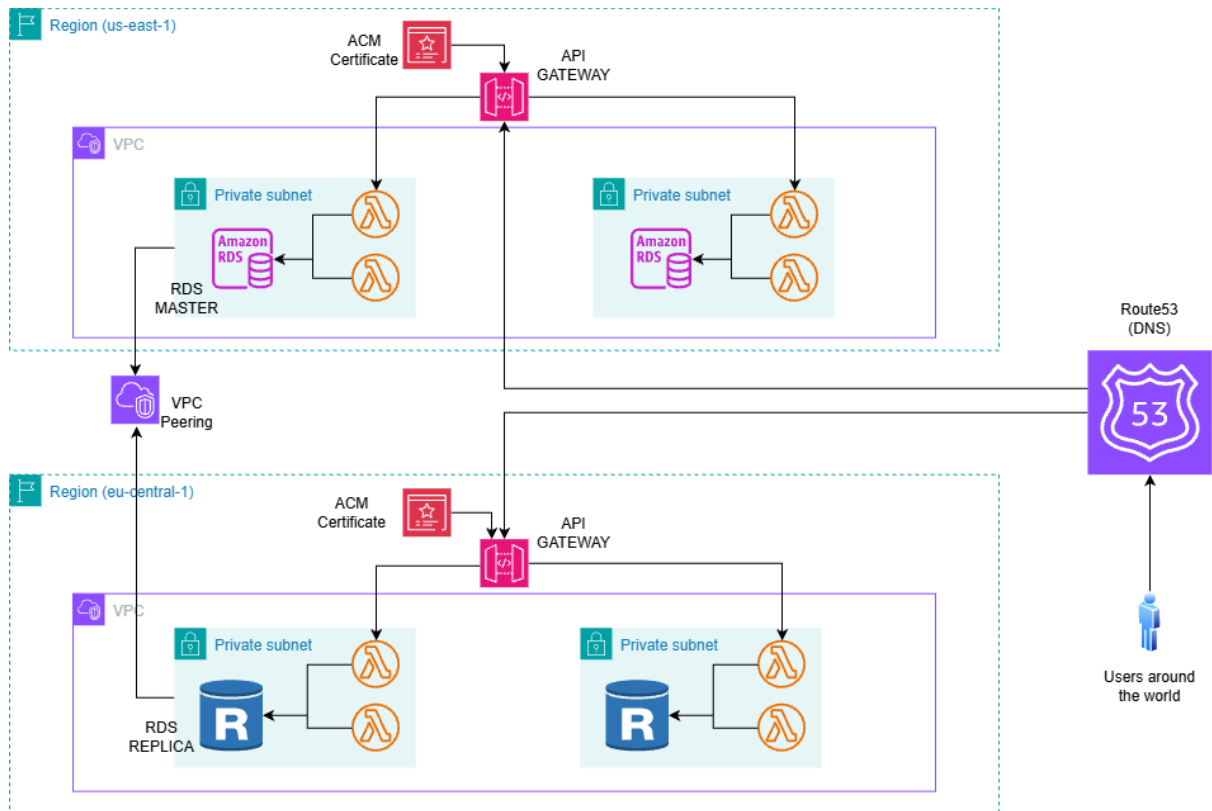
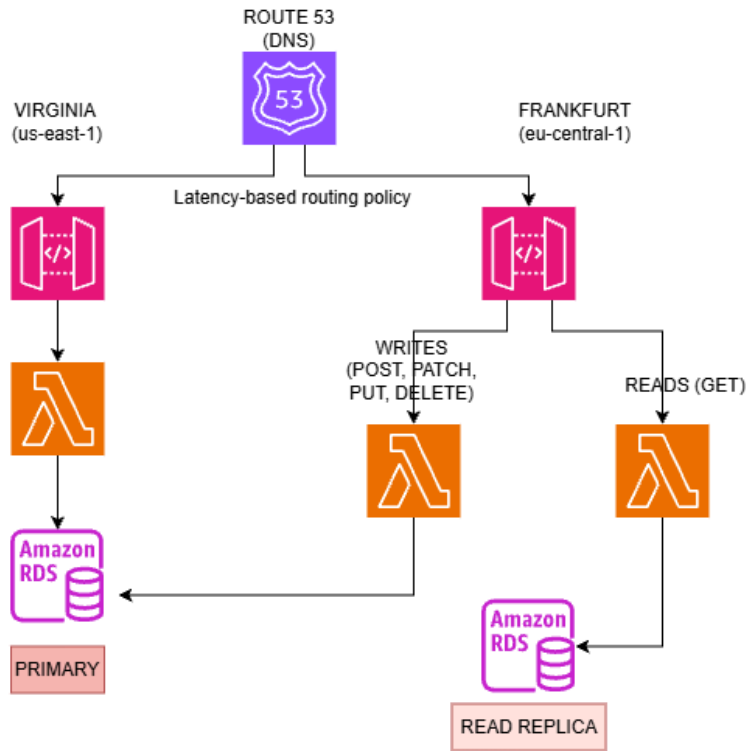
3.2. Multi-region

The second architecture added Frankfurt (eu-central-1) as an additional region.

The main changes were:

- A second API Gateway and Lambda deployment in Frankfurt
- An RDS read replica in Frankfurt
- Reads handled by the regional replica
- Writes still sent to the primary database in N. Virginia
- Route 53 latency-based routing used to direct users toward a regional API endpoint

This setup made it possible to compare the benefits of regionalizing reads while keeping a single write primary.



4. Test Environment

4.1. Test Locations

- North Virginia
- Frankfurt
- São Paulo
- Tokyo
- Sydney

4.2. Grafana Cloud K6

Grafana Cloud k6 was used to generate requests from different global load zones without manually deploying clients in each region.

4.3. Metrics

The main latency metrics collected were:

- Average latency
- p95
- p99
- Minimum latency
- Maximum latency

5. Results

5.1. Virginia Endpoint

- Read latency

Origin	avg	p95	p99	min	max
Virginia	64ms	62ms	~1s	21ms	1437ms
Frankfurt	146ms	150ms	~1s	110ms	1330ms
Sao Paulo	200ms	195ms	~1s	134ms	1474ms
Tokyo	201ms	216ms	540ms	173ms	546ms
Sydney	236ms	263ms	529ms	211ms	535ms

- Write latency

Origin	avg	p95	p99	min	max
Virginia	62ms	85ms	~1s	18ms	~1s
Frankfurt	129ms	137ms	464ms	110ms	470ms
Sao Paulo	146ms	160ms	374ms	134ms	378ms
Tokyo	186ms	199ms	399ms	166ms	403ms
Sydney	230ms	243ms	426ms	211ms	431ms

5.2. Frankfurt Endpoint

- Read latency

Origin	avg	p95	p99	min	max
Virginia	130ms	151ms	408ms	108ms	412ms
Frankfurt	34ms	44ms	382ms	16ms	386ms
Sao Paulo	241ms	266ms	529ms	215ms	535ms
Tokyo	262ms	289ms	518ms	240ms	523ms
Sydney	335ms	361ms	577ms	311ms	583ms

- Write latency

Origin	avg	p95	p99	min	max
Virginia	237ms	219ms	~2s	197ms	~2s
Frankfurt	130ms	122ms	928ms	108ms	939ms
Sao Paulo	331ms	329ms	~1s	311ms	~1s
Tokyo	361ms	368ms	~1s	332ms	~1s
Sydney	420ms	430ms	~1s	395ms	~1s

5.3. Route 53 Endpoint

- Read latency

Origin	avg	p95	p99	min	max
Virginia	56ms	78ms	~1s	16ms	~1s
Frankfurt	35ms	59ms	301ms	18ms	304ms
Sao Paulo	179ms	176ms	~2s	131ms	~2s
Tokyo	296ms	300ms	~2s	240ms	~2s
Sydney	267ms	277ms	~2s	215ms	~2s

- Write latency

Origin	avg	p95	p99	min	max
Virginia	52ms	47ms	~1s	17ms	~1s
Frankfurt	152ms	135ms	~2s	110ms	~2s
Sao Paulo	151ms	183ms	517ms	131ms	523ms
Tokyo	359ms	360ms	~1s	332ms	~1s
Sydney	240ms	265ms	529ms	220ms	535ms

6. Analysis

6.1. Geographic Distance

The first clear result is that geographic distance has a direct impact on backend latency.

When all components were hosted in N. Virginia, nearby users achieved the lowest latency, while users farther away experienced progressively higher response times. For reads against the Virginia endpoint, average latency increased from 64 ms in Virginia to 146 ms in Frankfurt, 200 ms in São Paulo, 201 ms in Tokyo, and 236 ms in Sydney.

This provides a practical measurement of the cost of serving a global user base from a single AWS region.

6.2. Regional Read Replica

The Frankfurt read replica produced one of the clearest improvements in the experiment.

A user in Frankfurt experienced an average read latency of 146 ms when accessing the Virginia endpoint. When the same request was served by the Frankfurt API, Lambda and RDS read replica, average latency dropped to 34 ms, an improvement of approximately 77%.

This validates the expected benefit of placing both compute and readable data closer to users.

6.3. Regional Compute And Remote Writes

Moving API Gateway and Lambda closer to users did not significantly improve writes when the primary database remained in Virginia.

Writes from Frankfurt averaged 129 ms through the Virginia endpoint and 130 ms through the Frankfurt endpoint.

In both cases, the request ultimately had to reach the primary database in Virginia. The network path changed, but the cross-region dependency remained.

This shows that moving compute alone does not necessarily reduce end-to-end latency when the main data dependency is still remote.

6.4. Inefficient Cross-Region Paths

Using the wrong regional endpoint can make writes significantly worse.

For a user in São Paulo, writes through the Virginia endpoint averaged 146 ms, while writes through Frankfurt increased to 331 ms.

The Frankfurt path introduced an unnecessary regional detour before reaching the primary database in Virginia:

São Paulo → Frankfurt → Virginia → Frankfurt → São Paulo.

This is a good example of why regional compute is not automatically beneficial if the data layer is located elsewhere.

6.5. Route53 Behavior

Route 53 latency-based routing generally selected reasonable regional endpoints, but the experiment also showed an important limitation.

Tokyo reads averaged 201 ms against Virginia, 262 ms against Frankfurt, and 296 ms through Route 53. Writes showed an even larger difference: 186 ms against Virginia compared with 359 ms through Route 53.

These results suggest that the regional endpoint selected by Route 53 was not always the best option for the complete application path.

Route 53 optimizes latency toward the regional endpoint, but it does not understand internal dependencies such as a Frankfurt Lambda needing to access a primary database in Virginia.

Sydney behaved more reasonably: writes averaged 230 ms through Virginia and 240 ms through Route 53, while Frankfurt reached 420 ms.

6.6. Two Regions Are Not Global

Adding Frankfurt significantly improved the experience for European users, but it did not make the architecture globally low-latency.

Users in São Paulo, Tokyo and Sydney still experienced relatively high latency. For example, read latency against Virginia was 200 ms in São Paulo, 201 ms in Tokyo, and 236 ms in Sydney.

A two-region architecture improves specific geographic areas, but a truly global system would require additional regional infrastructure closer to users in Asia, Oceania and South America.

6.7. Reads vs Writes

The experiment highlights a fundamental difference between distributing reads and writes.

Reads are relatively easy to move closer to users because read replicas can provide local copies of the data. This is why Frankfurt achieved such a large improvement in read latency.

Writes are more difficult. In a single-primary architecture, every mutation still depends on the location of the primary database. As a result, write latency remains strongly tied to the writer region, regardless of where API Gateway or Lambda are deployed.

7. Limitations

The experiment was intentionally focused on geographic latency rather than throughput or system capacity. Each test run lasted one minute, used a single virtual user, and generated 50–60 requests. Because of the relatively small sample size, p99 values were highly sensitive to isolated slow requests and should be interpreted carefully.

Cold starts were not separated from normal execution latency, so some of the maximum values likely include Lambda initialization or initial database connection overhead. The architecture also used only two AWS regions, N. Virginia and Frankfurt, which is not enough to represent a truly global deployment.

The experiment did not measure RDS replication lag, and no load or stress testing was performed. Therefore, the results describe latency under very light traffic conditions, not system behavior under concurrency or saturation.

The experiment did not compare other global architectures or routing mechanisms such as AWS Global Accelerator, Aurora Global Database, or more advanced multi-region write strategies.

8. Conclusions

The experiment confirmed that geographic distance has a clear impact on API latency. Regional read replicas significantly improved read performance for European users, while writes remained constrained by the location of the primary database. It also showed that adding regional compute does not automatically improve end-to-end latency if downstream dependencies remain remote.

Route 53 latency-based routing helped in several cases, but the experiment also showed that the closest regional endpoint is not always the best end-to-end path. Overall, the results reinforce that multi-region architectures must be designed around data placement and access patterns, not just around where compute is deployed.